



UNIVERSIDAD TECNOLÓGICA METROPOLITANA
FACULTAD DE INGENIERÍA
DEPARTAMENTO DE INFORMÁTICA Y COMPUTACIÓN

Un problema de cuadratura.

Integrantes: Alan Opazo; Ailyn Strajilevich; Cristobal Muñoz

Profesor: Sebastián Salazar Molina

Fecha de entrega: 15 junio 2026

Índice:

- Introducción
- Descripción de la solución implementada
- Pseudocódigo del algoritmo paralelo
- Análisis de rendimiento (tiempos de ejecución, speedup con OpenMP)
- Dificultades encontradas y cómo se resolvieron
- Conclusión

Introducción:

La cadena de farmacias Cruz Morada, líder en el mercado chileno, enfrenta desafíos en el análisis eficiente de sus datos de ventas. Actualmente, los registros se almacenan en archivos CSV en un servidor SFTP, siguiendo una nomenclatura estandarizada. Cada archivo contiene miles de transacciones diarias, y el volumen de datos crece rápidamente.

El objetivo de este trabajo es desarrollar una solución en C/C++ que, utilizando paralelismo con OpenMP, procese estos archivos y consulte de forma masiva una API REST protegida para calcular métricas clave: el promedio de compras realizadas por mujeres y por hombres. Este ejercicio simula un escenario real en el que el rendimiento y la eficiencia son críticos para la toma de decisiones empresariales.

Descripción de la solución implementada:

La solución desarrollada se fundamenta en una arquitectura de alto rendimiento escrita en C++, utilizando el estándar OpenMP para el procesamiento paralelo y la biblioteca libcurl para las peticiones de red. El diseño aborda el desafío como un problema fuertemente limitado por entrada/salida (I/O Bound), donde el cuello de botella no es la capacidad de procesamiento de la CPU, sino la latencia de la red al consumir la API REST.

A continuación, se detalla la justificación técnica de cada componente de la solución:

1. Gestión Concurrente de Autenticación (AuthManager):

Se implementó una clase encargada de obtener y renovar el token JWT. Dado que cientos de hilos pueden recibir un error de autorización (HTTP 401/403) simultáneamente si el token expira, se protegió la variable del token utilizando un `std::shared_mutex` (candado de lectura/escritura). Esto permite que múltiples hilos lean el token de forma concurrente sin bloqueos, pero asegura exclusión mutua estricta cuando un hilo necesita escribir y renovar el token, evitando condiciones de carrera (race conditions).

2. Optimización de Transferencia de Archivos (SFTPManager):

Para evitar el costo computacional y de tiempo de descargar repetidamente los archivos, se implementó una función de sincronización inteligente. El algoritmo solicita el listado de archivos al servidor SFTP y verifica su existencia y peso a nivel local (`std::filesystem`). Solo se descargan mediante hilos concurrentes de OpenMP aquellos archivos que faltan o están corruptos, ahorrando ancho de banda y reduciendo drásticamente los tiempos de preparación del entorno.

3. Parseo Paralelo de Datos (DataProcessor - Parseo):

La lectura de los archivos CSV se distribuyó dinámicamente (`schedule(dynamic)`) entre el número máximo de hilos disponibles por la CPU. Cada hilo extrae la información y la acumula en un mapa hash local (`std::unordered_map`). Al finalizar su bloque de archivos, cada hilo ingresa a una sección crítica (`#pragma omp critical`) únicamente para fusionar sus mapas

locales en un mapa global. Esta técnica (reducción local) elimina la necesidad de bloquear el mapa global por cada línea leída, maximizando el uso del procesador.

4. Concurrencia Extrema y Optimización de Red (Over-subscription):

La consulta a la API de validación representaba el mayor cuello de botella del sistema para los casi 3 millones de registros. Para solucionarlo, se implementaron las siguientes técnicas de grado de producción:

- **Over-subscription (600 Hilos):** Se asignaron 600 hilos concurrentes de OpenMP, superando ampliamente el número de núcleos físicos del procesador. Se justifica porque las consultas HTTP bloquean el hilo a la espera de red, por lo que la CPU delega el tiempo de espera y administra eficientemente cientos de peticiones sin saturar el procesador.
- **Túneles Persistentes (TCP Keep-Alive):** Se inyectó la directiva `CURLOPT_TCP_KEEPALIVE` en `cURL`. Esto evita realizar el costoso protocolo de negociación de seguridad (SSL Handshake) por cada cliente procesado, manteniendo los túneles abiertos para solicitudes continuas y continuadas.
- **Balanceo Dinámico de Carga (Chunking):** Se aplicó un `#pragma omp for schedule(dynamic, 50)`. Repartir la carga en lotes de 50 registros evita que la CPU se sobrecargue administrando asignaciones individuales.

5. Mitigación de Overhead en Consola (Bloqueador de I/O) En sistemas altamente paralelos, obligar a múltiples hilos a escribir constantemente en la salida estándar (pantalla) genera un embotellamiento crítico, obligando a los hilos a hacer cola para imprimir. Se mitigó esta latencia exigiendo que cada hilo procese individualmente 500 clientes en memoria antes de solicitar el bloqueo de pantalla para imprimir el progreso global. Los remanentes de clientes procesados se suman de forma segura al finalizar el ciclo mediante un bloque `#pragma omp critical(acumulado_final)`.

Pseudocódigo del algoritmo paralelo:

INICIO PROGRAMA

Registrar tiempo_inicio_reloj

// 1. VALIDACIÓN INICIAL

SI faltan variables de entorno (API_EMAIL, API_RUT, SFTP_CREDS) ENTONCES

TERMINAR PROGRAMA con error

FIN SI

Crear directorio local "data" si no existe

// 2. AUTENTICACIÓN

token_jwt = ObtenerTokenAPI(API_EMAIL, API_RUT)

SI token_jwt es nulo ENTONCES

TERMINAR PROGRAMA con error

FIN SI

// 3. DESCARGA DESDE SFTP

lista_archivos_faltantes = ConsultarSFTP_ArchivosAusentes(directorio "data")

SI lista_archivos_faltantes NO está vacía ENTONCES

Configurar OpenMP(hilos = 4)

INICIO BLOQUE PARALELO:

Por cada archivo en lista_archivos_faltantes:

Descargar archivo CSV y guardar en "data"

FIN BLOQUE PARALELO

FIN SI

// 4. PARSEO DE ARCHIVOS CSV (Mapeo Local)

Diccionario global_clientes

lista_csv = ObtenerTodosLosArchivosCSV("data")

Configurar OpenMP(hilos = MAX_HILOS_CPU)

INICIO BLOQUE PARALELO (Dinámico):

Diccionario local_clientes

Por cada archivo en lista_csv asignado al hilo:

Abrir archivo

Leer línea por línea saltando encabezado

Extraer UUID y Monto_Total

Acumular Monto_Total y sumar +1 transacción en local_clientes[UUID]

INICIO SECCIÓN CRÍTICA:

Fusionar local_clientes dentro de global_clientes

FIN SECCIÓN CRÍTICA

FIN BLOQUE PARALELO

// 5. CONSULTA A LA API (Optimización Extrema de Red)

Métricas global_metricas = (0, 0, 0, 0)

Contador_Progreso = 0

total_clientes = Tamaño(global_clientes)

Configurar OpenMP(hilos = 600) // Máxima concurrencia

INICIO BLOQUE PARALELO (Chunking dinámico en bloques de 50):

```
Métricas local_metricas = (0, 0, 0, 0)
procesados_por_hilo = 0
```

```
Configurar Cliente_HTTP:
```

```
    Activar TCP_KEEPALIVE
```

```
    Activar Aceptación_GZIP
```

```
    Desactivar Verificación_SSL_Host
```

```
Por cada cliente (UUID, info) en global_clientes asignado al hilo:
```

```
    intentos = 0
```

```
    genero_obtenido = "NO_DEFINIDO"
```

```
MIENTRAS intentos < 2 HACER:
```

```
    respuesta = Ejecutar_Peticion_GET(URL + UUID, token_jwt)
```

```
    SI código_HTTP == 200 ENTONCES
```

```
        genero_obtenido = Extraer_Genero_De_JSON(respuesta)
```

```
        ROMPER CICLO
```

```
    SINO SI código_HTTP == 401 Ó 403 ENTONCES
```

```
        Renovar_Token_JWT() // Protegido por mutex de lectura/escritura
```

```
        intentos++
```

```
    SINO
```

```
        ROMPER CICLO
```

```
    FIN SI
```

```
FIN MIENTRAS
```

```
SI genero_obtenido == "FEMENINO" ENTONCES
```

```
    local_metricas.suma_fem += info.monto
```

```
    local_metricas.cuenta_fem += info.transacciones
```

```
SINO SI genero_obtenido == "MASCULINO" ENTONCES
```

```
    local_metricas.suma_masc += info.monto
```

```
    local_metricas.cuenta_masc += info.transacciones
```

```
FIN SI
```

```
procesados_por_hilo++
```

```
// Bloqueador de sobrecarga de consola
```

```
SI procesados_por_hilo módulo 500 == 0 ENTONCES
```

```
    INICIO SECCIÓN CRÍTICA:
```

```
        Contador_Progreso += procesados_por_hilo
```

```
        procesados_por_hilo = 0
```

```
        Imprimir_En_Pantalla(Contador_Progreso)
    FIN SECCIÓN CRÍTICA
FIN SI
```

```
// Fin del ciclo del hilo
INICIO SECCIÓN CRÍTICA:
    Fusionar local_metricas en global_metricas
    Contador_Progreso += procesados_por_hilo // Sumar los remanentes
FIN SECCIÓN CRÍTICA
```

FIN BLOQUE PARALELO

```
// 6. CÁLCULO DE RESULTADOS FINALES
tiempo_total = Obtener_Tiempo_Actual() - tiempo_inicio_reloj
promedio_fem = global_metricas.suma_fem / global_metricas.cuenta_fem
promedio_masc = global_metricas.suma_masc / global_metricas.cuenta_masc

Imprimir Resultados: promedio_fem, promedio_masc, tiempo_total
Escribir Resultados en archivo "resultados.txt"
```

FIN PROGRAMA

Análisis de rendimiento (tiempos de ejecución, speedup con OpenMP):

1. Tiempos de Ejecución y Cálculo de Speedup El diseño original del consumo de la API de forma secuencial presentaba un tiempo de ejecución proyectado y prohibitivo de aproximadamente 111 horas (399.600 segundos) para procesar los casi 3 millones de clientes.

Tras la implementación de la arquitectura paralela con OpenMP (optimizada a 600 hilos y conexiones TCP persistentes), el programa logró procesar la totalidad de los datos en 25610.27 segundos (aproximadamente 7.11 horas).

Aplicando la ley de Amdahl para medir la aceleración, calculamos el *Speedup* (S) obtenido:

$$S = T_{\text{secuencial}} / T_{\text{paralelo}}$$

$$S = 399600 \text{ segundos} / 25610.27 \text{ segundos} \approx 15.6$$

El sistema logró un Speedup de 15.6x, lo que significa que la ejecución paralela es más de 15 veces más rápida que el enfoque secuencial, transformando un problema de días en uno de horas.

2. Comportamiento de Recursos del Sistema

Durante la ejecución a máxima concurrencia, las métricas del contenedor evidenciaron un comportamiento estable de grado de producción:

- **Uso de Memoria (Memory Usage):** La carga del diccionario `std::unordered_map` en memoria RAM se estabilizó tempranamente en 1.62 GB y se mantuvo constante durante las 7 horas de ejecución. La línea plana del gráfico de memoria demuestra empíricamente la ausencia de *memory leaks* en el código C++.
- **Ancho de Banda (Network I/O):** El gráfico de transferencia muestra una pendiente positiva constante, sin interrupciones, procesando cientos de megabytes en peticiones asíncronas de red de manera fluida y sin bloqueos de servidor.
- **Uso de Procesador (CPU Usage):** A pesar de operar con 600 hilos, la CPU mantuvo un promedio de uso inferior al 30%. Esto confirma de manera concluyente que el problema es del tipo *I/O Bound* (limitado por red) y que el paralelismo masivo fue absorbido eficientemente delegando el tiempo de espera a la red, sin saturar los recursos de procesamiento.

Dificultades encontradas y cómo se resolvieron:

1. Cuello de botella I/O en la descarga de archivos (Limitación de Máquina Virtual):

Durante la fase inicial de ingesta de datos, la descarga de los 913 archivos CSV desde el servidor SFTP presentó un severo cuello de botella debido a las limitaciones de la máquina virtual Ubuntu empleada, la cual contaba con una asignación restringida de solo 4 hilos de CPU. Esto asfixiaba la capacidad de procesamiento de red y las operaciones de entrada/salida (I/O) en disco. Para superar esta limitante arquitectónica, se abandonó la virtualización tradicional en favor de la contenerización nativa con Docker. Esta migración permitió aprovechar el rendimiento *bare-metal* del procesador anfitrión (Apple Silicon) e implementar un volumen montado de alto rendimiento (Bind mount), facilitando la sincronización directa de los archivos en el almacenamiento físico del equipo a máxima velocidad y erradicando el embotellamiento inicial.

2. Desconocimiento arquitectónico en el consumo de la API REST y Autenticación:

Un segundo obstáculo significativo fue la falta de experiencia práctica respecto al diseño y consumo de APIs REST protegidas, lo que resultó en el rechazo sistemático de las peticiones

iniciales (HTTP 401/403) al no gestionar adecuadamente el flujo de autenticación *stateless* (sin estado). Para dar solución a esta deficiencia, se investigó el estándar JSON Web Tokens (JWT) y se desarrolló la clase abstracta AuthManager. Esta capa utiliza libcurl para construir un *payload* JSON con las credenciales, ejecutar una petición HTTP POST al endpoint de autenticación y extraer el token dinámico de la respuesta. Posteriormente, este token se inyecta de manera automática en las cabeceras HTTP (Authorization: Bearer <token>) de cada uno de los cientos de hilos, asegurando un acceso validado y concurrente bajo candados de lectura/escritura (std::shared_mutex).

3. Incompatibilidad del compilador y soporte nativo de OpenMP en macOS:

A nivel de desarrollo, el equipo enfrentó errores críticos de compilación ('omp.h' file not found y dependencias indefinidas de funciones como omp_get_wtime) al intentar construir el código en el entorno nativo de macOS. Estos fallos derivan de la incompatibilidad del compilador por defecto de Apple (Apple Clang), el cual carece de soporte nativo preinstalado para el estándar OpenMP. La resolución integral de este problema se logró mediante la orquestación del contenedor Docker basado en Ubuntu 24.04, garantizando un entorno Linux estandarizado con GCC. Adicionalmente, se debió reestructurar el archivo Makefile para forzar la inclusión estricta de las directivas -fopenmp tanto en la fase de compilación (CXXFLAGS) como en el enlazado (LDFLAGS), permitiendo al compilador resolver y orquestar exitosamente las macros #pragma.

4. Latencia por negociación criptográfica (SSL Handshake) y escalado de hilos:

En la fase de consumo masivo de la API, se detectó una latencia extrema que proyectaba más de 10 horas de ejecución incluso utilizando 150 hilos concurrentes. El análisis de rendimiento reveló que el problema radicaba en el *SSL Handshake*: cada hilo abría y cerraba una nueva conexión segura HTTPS por cada cliente, multiplicando la sobrecarga de red. Para subsanar esta ineficiencia, se aplicó una optimización agresiva inyectando la directiva CURLOPT_TCP_KEEPALIVE en cURL, forzando al servidor a mantener cientos de túneles TCP bidireccionales abiertos de forma persistente. Reconociendo que el problema era inherentemente *I/O Bound* (limitado por red, no por CPU), se escaló la arquitectura a la capacidad extrema de 600 hilos OpenMP, logrando saturar la capacidad de respuesta del servidor de forma controlada y reduciendo el tiempo de procesamiento a poco más de 7 horas.

5. Embotellamiento de I/O en la Consola (Lock Contention):

Finalmente, la inserción de 600 hilos concurrentes desencadenó un fenómeno conocido como *Lock Contention* (condición de competencia) sobre el flujo de salida estándar

(consola). Al intentar imprimir el progreso (`std::cout`) de forma individual, los hilos saturaban al planificador del sistema operativo haciendo fila para obtener exclusión mutua sobre la terminal, degradando severamente el rendimiento general. La estrategia de mitigación consistió en diseñar un agrupador de memoria local condicional (`if procesados % 500 == 0`). De este modo, cada hilo procesa y acumula resultados de red en silencio, interrumpiendo a la CPU para imprimir en pantalla únicamente tras completar bloques densos de 500 registros, técnica que redujo el *overhead* de sincronización a un nivel imperceptible.

Conclusión:

El desarrollo de esta solución ha demostrado de manera empírica el impacto crítico que tiene la arquitectura de software y la programación paralela en el procesamiento masivo de datos. Se logró cumplir exitosamente con el objetivo de calcular las métricas comerciales para la cadena de farmacias Cruz Morada, transformando un problema secuencial prohibitivo, proyectado en 111 horas de ejecución, en un proceso altamente eficiente de apenas 7.11 horas. Este salto cualitativo, materializado en un *Speedup* de 15.6x, no fue producto exclusivo de aumentar la fuerza bruta computacional, sino de un análisis profundo del comportamiento de los recursos del sistema.

El hallazgo técnico más relevante del proyecto fue identificar y abordar que el verdadero cuello de botella no residía en la capacidad de cálculo de la CPU, sino en la latencia de red, catalogando el desafío como un problema estrictamente *I/O Bound*. Gracias a esta comprensión, se superó el paradigma tradicional de limitar la concurrencia a los núcleos físicos del procesador, implementando una arquitectura de *Over-subscription* con 600 hilos mediante OpenMP. La combinación de este paralelismo masivo con optimizaciones de red puras —como la persistencia de túneles mediante TCP Keep-Alive para erradicar el *overhead* del SSL Handshake— permitió delegar los tiempos de espera a la red de forma concurrente. Esto quedó evidenciado al mantener el uso del procesador por debajo del 30% en su máxima carga de trabajo.

Adicionalmente, el diseño descentralizado del parseo, utilizando reducciones locales de diccionarios Hash (`std::unordered_map`), sumado a la mitigación del embotellamiento de I/O en la terminal (Lock Contention), garantizó una ejecución fluida e ininterrumpida. El sistema exhibió una estabilidad de grado de producción, manteniendo el consumo de memoria RAM plano en 1.62 GB. Esta métrica corrobora definitivamente la solidez del código en C++ y la total ausencia de fugas de memoria (*memory leaks*) ante un estrés de millones de transacciones.

En síntesis, este trabajo ratifica que, en la ingeniería de software moderna, el máximo rendimiento se alcanza cuando convergen el paralelismo a nivel de procesador, la optimización asíncrona de redes y la gestión estricta de memoria. Para un entorno corporativo real, la aplicación exitosa de estos principios de concurrencia se traduce

directamente en la reducción de costos en infraestructura y en la agilización de la inteligencia de negocios para la toma de decisiones estratégicas.